

CAHIER DES CHARGES

Projet d'implémentation de la commande lsof en langage C

Nom du projet : MYLSOF

Comprendre Linux par la pratique : lire les processus, analyser les descripteurs de fichiers et reconstruire une commande système utile.

Membres du groupe:

- BAYI NGANTCHEU Ange Franka
- KENGNE TAKOUMBO Ange Naomy
- NTCHANTCHOU NGASSA Sobrin
- SARR SALIF
- TIENTCHEU KAMGA Orthys Cassandra

Année Scolaire: 2025- 2026

CLASSE: ING 3SRT

SUPERVISEUR: M. NGUIMBUS

Sommaire

- 1. Contexte du projet
- 2. Présentation de la commande lsof et du projet mylsof
- 3. Objectifs du projet
- 4. Périmètre fonctionnel
- 5. Périmètre technique
- 6. Contraintes, hypothèses et limites
- 7. Exigences non fonctionnelles
- 8. Organisation du code et architecture proposée
- 9. Stratégie de tests et validation
- 10. Livrables attendus
- 11. Plan de travail
- 12. Critères d'acceptation
- 13. Risques du projet et mesures de maîtrise
- 14. Références
- Annexes

NB : ce sommaire est volontairement simple afin de rester propre dans Word. Il peut être remplacé par une table des matières automatique avant dépôt final.

1. Contexte du projet

Dans les systèmes Linux, une grande partie des informations liées aux processus, aux fichiers ouverts et aux ressources utilisées est accessible à travers des commandes système. Parmi ces commandes, lsof occupe une place importante, car elle permet d'observer quels fichiers sont ouverts, par quels processus et avec quels descripteurs.

Le projet mylsof s'inscrit dans une démarche d'apprentissage par la pratique. L'objectif n'est pas seulement de recopier le comportement d'une commande existante, mais de comprendre comment Linux expose ses données internes, comment un programme C peut les lire proprement, et comment transformer ces informations brutes en un affichage exploitable par un utilisateur.

Ce cahier des charges reprend l'esprit du modèle fourni pour le projet mygrep : un contexte clair, des objectifs, un périmètre fonctionnel, des contraintes techniques, des livrables et un plan de travail. Le contenu a toutefois été adapté à la logique de lsof, qui repose principalement sur l'exploration des processus et du système de fichiers /proc.

2. Présentation de la commande lsof et du projet mylsof

La commande lsof signifie « list open files ». Elle sert à lister les fichiers ouverts par les processus actifs du système. Dans Linux, la notion de fichier est très large : un fichier peut être un document classique, un dossier, un tube, un socket réseau, un périphérique, une bibliothèque chargée ou encore le répertoire courant d'un processus.

Le projet mylsof consistera donc à développer, en langage C, une version simplifiée mais fonctionnelle de lsof. Le programme devra parcourir les processus visibles dans /proc, identifier leurs descripteurs de fichiers ouverts, récupérer les informations utiles, puis les afficher dans un format lisible et proche de la commande originale.

Commande cible :

`./mylsof [OPTIONS] [FILTRE]`

Exemple attendu :

- `./mylsof`
- `./mylsof -p 1234`
- `./mylsof -u axel`
- `./mylsof -c ssh`
- `./mylsof /var/log/syslog`

3. Objectifs du projet

3.1 Objectif général

L'objectif principal du projet est de mettre sur pied un clone pédagogique de la commande lsof, nommé mylsof, en langage C, capable d'afficher les fichiers ouverts par les processus d'un système Linux.

3.2 Objectifs spécifiques

- **Comprendre le fonctionnement de /proc** : identifier les répertoires représentant les processus, lire leurs fichiers d'information et exploiter les liens symboliques présents dans `/proc/[pid]/fd`.
- **Manipuler les processus** : récupérer le PID, le nom de la commande, l'utilisateur propriétaire et les informations associées.

- **Lire les descripteurs de fichiers** : déterminer les fichiers, répertoires, pipes ou sockets ouverts par chaque processus accessible.
- **Développer une interface en ligne de commande** : gérer des options simples, afficher une aide, filtrer les résultats et traiter les erreurs.
- **Produire un code propre** : organiser le programme en modules, utiliser un Makefile, commenter les fonctions importantes et publier le projet sur GitHub.
- **Valider le résultat** : comparer le comportement de mylsf avec lsof sur des cas simples, tout en acceptant que mylsf reste une version pédagogique et non une copie exhaustive.

4. Périmètre fonctionnel

Le périmètre fonctionnel définit ce que mylsf doit faire, ce qui est prioritaire et ce qui pourra rester hors périmètre. L'idée est d'obtenir une version suffisamment complète pour être utile, tout en restant réaliste pour un projet étudiant.

4.1 Fonctionnement général attendu

- Si aucune option n'est fournie, mylsf affiche les fichiers ouverts par les processus accessibles à l'utilisateur courant.
- Le programme parcourt les répertoires numériques présents dans /proc, car ils correspondent aux PID des processus actifs.
- Pour chaque processus, le programme lit les informations principales : PID, commande, UID/utilisateur, état si disponible.
- Pour chaque processus, le programme explore /proc/[pid]/fd afin de récupérer les descripteurs de fichiers ouverts.
- Les résultats sont affichés sous forme de tableau lisible, avec des colonnes proches de lsof.
- Les erreurs de permission ne doivent pas interrompre tout le programme : elles sont ignorées proprement ou signalées selon le mode choisi.

4.2 Colonnes d'affichage prévues

Colonne	Signification	Source possible
COMMAND	Nom de la commande ou du programme associé au processus.	/proc/[pid]/comm ou /proc/[pid]/cmdline
PID	Identifiant numérique du processus.	Nom du répertoire /proc/[pid]
USER	Utilisateur propriétaire du processus.	UID dans /proc/[pid]/status puis correspondance via getpwuid()
FD	Descripteur de fichier ouvert : 0, 1, 2, cwd, exe, etc.	/proc/[pid]/fd, /proc/[pid]/cwd, /proc/[pid]/exe
TYPE	Type de ressource : fichier régulier, dossier, socket, pipe, périphérique, inconnu.	stat(), lstat(), analyse du lien symbolique
DEVICE	Identifiant du périphérique lorsque disponible.	st_dev dans struct stat
SIZE/OFF	Taille du fichier ou information d'offset si disponible.	st_size, données partielles selon faisabilité
NODE	Numéro d'inode si disponible.	st_ino dans struct stat
NAME	Chemin ou nom de la ressource ouverte.	readlink() sur /proc/[pid]/fd/[fd]

4.3 Fonctionnalités prioritaires

ID	Fonctionnalité	Description	Priorité
F01	Affichage global	Lister les fichiers ouverts des processus accessibles sans option particulière.	Haute
F02	Aide utilisateur	Afficher une aide claire avec --help et expliquer les options	Haute

		disponibles.	
F03	Version	Afficher le nom du projet, la version, les auteurs et le lien GitHub avec --version.	Haute
F04	Filtre par PID	Afficher uniquement les fichiers ouverts par un processus donné : -p PID ou --pid PID.	Haute
F05	Filtre par utilisateur	Afficher uniquement les processus appartenant à un utilisateur : -u USER ou --user USER.	Haute
F06	Filtre par commande	Filtrer selon le nom de la commande : -c NOM ou --command NOM.	Moyenne
F07	Filtre par fichier/chemin	Afficher les processus ayant ouvert un chemin donné.	Moyenne
F08	Mode silencieux des erreurs	Masquer les messages liés aux permissions ou fichiers disparus : -s ou --silent.	Moyenne
F09	Affichage PID seul	Afficher uniquement les PID concernés : -t ou --terse.	Moyenne
F10	Sortie sans en-tête	Désactiver l'en-tête du tableau : --no-header.	Basse

4.4 Fonctionnalités secondaires ou bonus

Option	Usage proposé	Intérêt pédagogique
-d FD / --fd FD	Filtrer un descripteur précis, par exemple 0, 1, 2 ou 3.	Comprendre le rôle des descripteurs.
+D DOSSIER	Lister les fichiers ouverts dans un dossier de manière récursive.	Travailler la comparaison de chemins et la récursivité.
--json	Exporter les résultats en JSON.	Préparer le programme à une exploitation automatique.
--stats	Afficher un résumé : nombre de processus analysés, erreurs, fichiers trouvés.	Donner une vision de contrôle au développeur.
--color	Colorer certains types de ressources en sortie terminal.	Améliorer le confort de lecture.

4.5 Fonctionnalités hors périmètre

Certaines options avancées de lsof ne seront pas exigées dans la première version, car elles demandent une analyse réseau ou noyau plus poussée. Elles pourront être mentionnées comme limites du projet.

- Analyse complète des sockets réseau avec résolution d'adresses, ports et protocoles avancés.
- Compatibilité macOS, BSD ou Windows.
- Reproduction exacte de toutes les colonnes et options de la commande lsof officielle.
- Gestion exhaustive des fichiers mappés en mémoire, bibliothèques partagées et fichiers supprimés encore ouverts.
- Prise en charge complète des namespaces, conteneurs et environnements restreints.

4.6 Cas d'utilisation

Cas	Commande attendue	Résultat souhaité
Lister les fichiers ouverts	./mylsof	Affiche un tableau de tous les fichiers ouverts accessibles.
Analyser un processus précis	./mylsof -p 1234	Affiche seulement les fichiers ouverts par le PID 1234.
Trouver les fichiers ouverts par un utilisateur	./mylsof -u axel	Affiche les processus appartenant à l'utilisateur indiqué.
Rechercher un programme	./mylsof -c bash	Affiche les entrées liées aux commandes contenant ou correspondant à bash.
Savoir qui utilise un fichier	./mylsof /chemin/fichier	Affiche les processus ayant ce fichier ouvert, si visible.
Utiliser dans un script	./mylsof -t /var/log/syslog	Affiche seulement les PID pour faciliter l'automatisation.

5. Périmètre technique

5.1 Environnement de développement

Élément	Choix retenu	Justification
Langage	C	Langage bas niveau adapté à la programmation système sous Linux.
Système cible	Linux	Le projet repose sur le système de fichiers /proc.
Compilateur	GCC	Outil courant, disponible sur la majorité des distributions Linux.
Gestion de projet	Git + GitHub	Suivi de versions, partage du code et dépôt final.
Compilation	Makefile	Compilation reproductible avec make, make clean et make test.
Tests	Scripts shell + comparaisons avec lsOF	Validation simple et démonstration rapide.
Documentation	README.md + rapport technique	Explication de l'installation, de l'usage et des choix de conception.

5.2 Sources de données Linux utilisées

Le programme n'interrogera pas directement le noyau avec une API complexe. Il s'appuiera principalement sur /proc, qui expose les informations des processus sous forme de fichiers et de répertoires.

Chemin	Utilisation dans mylsOF
/proc	Point d'entrée pour lister les processus actifs.
/proc/[pid]	Répertoire associé à un processus.
/proc/[pid]/fd	Liste des descripteurs de fichiers ouverts par le processus.
/proc/[pid]/fd/[fd]	Lien symbolique vers la ressource réellement ouverte.
/proc/[pid]/comm	Nom court de la commande.
/proc/[pid]/cmdline	Commande complète et arguments, lorsque disponibles.
/proc/[pid]/status	Informations textuelles comme UID, état et nom du processus.
/proc/[pid]/cwd	Répertoire courant du processus.
/proc/[pid]/exe	Lien vers l'exécutable du processus.
/etc/passwd ou base utilisateur	Correspondance UID -> nom utilisateur via les fonctions système.

5.3 Fonctions et appels système envisagés

Fonction / bibliothèque	Rôle dans le projet
opendir(), readdir(), closedir()	Parcourir /proc et /proc/[pid]/fd.
readlink()	Résoudre les liens symboliques vers les fichiers réellement ouverts.
stat(), lstat()	Récupérer le type, la taille, le device et l'inode d'une ressource.
getpwuid(), getpwnam()	Convertir les UID en noms d'utilisateurs et filtrer par utilisateur.
getopt_long()	Analyser proprement les options courtes et longues.
snprintf()	Construire des chemins sûrs sans dépassement de tampon.
errno, perror()	Gérer les erreurs et les permissions refusées.
malloc(), free()	Gérer les chaînes et structures dynamiques lorsque nécessaire.

5.4 Format de sortie attendu

La sortie standard doit rester lisible. L'objectif n'est pas d'être parfaitement identique à lsOF, mais de proposer un affichage cohérent et exploitable.

```
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME
bash     1842  salif  cwd  DIR    8,1    4096    262145  /home/salif
bash     1842  sobrin  1  REG    8,1      0    263001  /dev/pts/0
1021   root    3  IPv4  -      -      -      -  socket:[12345]
```

6. Contraintes, hypothèses et limites

6.1 Contraintes techniques

- **Système Linux obligatoire** : mylsof dépend de /proc, donc le programme ne vise pas Windows et ne garantit pas le même comportement sur macOS.
- **Droits d'accès** : certains processus appartiennent à root ou à d'autres utilisateurs. Le programme doit accepter les erreurs de permission sans se bloquer.
- **Processus dynamiques** : un processus peut disparaître pendant l'analyse. Le programme doit gérer les fichiers introuvables ou devenus illisibles.
- **Performance** : un système peut avoir beaucoup de processus et de descripteurs. Le parcours doit rester raisonnable et éviter les traitements inutiles.
- **Gestion mémoire** : les buffers et allocations doivent être maîtrisés pour éviter fuites, dépassements ou comportements indéfinis.
- **Conformité partielle** : mylsof est un clone pédagogique : il ne couvrira pas toutes les options de lsof officiel.

6.2 Hypothèses de travail

- Le programme sera testé sur une distribution Linux récente avec /proc monté.
- L'utilisateur exécutera la commande depuis un terminal.
- La première version se concentrera sur les processus visibles par l'utilisateur courant.
- Les comparaisons avec lsof serviront de repère, mais pas de contrainte de reproduction exacte.
- Le dépôt GitHub contiendra les sources, le Makefile, la documentation et les exemples de test.

7. Exigences non fonctionnelles

Catégorie	Exigence
Lisibilité	Les résultats doivent être alignés et compréhensibles, même quand certaines informations sont manquantes.
Robustesse	Une erreur sur un processus ne doit pas interrompre l'analyse de tous les autres.
Maintenabilité	Le code doit être découpé en modules avec des noms de fonctions explicites.
Traçabilité	Chaque version importante doit être suivie dans Git avec des commits clairs.
Portabilité limitée	Le programme doit fonctionner sur Linux standard, sans dépendance externe lourde.
Sécurité	Le programme ne doit pas modifier /proc, ni écrire dans les fichiers inspectés.
Performance	Le temps d'exécution doit rester acceptable sur une machine personnelle classique.
Pédagogie	Les choix techniques doivent être expliqués dans le rapport afin de montrer la compréhension du système.

8. Organisation du code et architecture proposée

Pour éviter un programme trop long dans un seul fichier, mylsof sera organisé en modules. Cette organisation rendra le projet plus facile à développer, à tester et à présenter lors de la démonstration.

Fichier / dossier	Rôle proposé
src/main.c	Point d'entrée : lance le parsing des options, l'analyse et l'affichage.
src/options.c / include/options.h	Gestion des options de la ligne de commande.
src/proc_scan.c / include/proc_scan.h	Parcours de /proc et identification des PID.
src/process_info.c / include/process_info.h	Lecture de comm, cmdline, status et informations utilisateur.
src/fd_info.c / include/fd_info.h	Lecture des descripteurs de fichiers et résolution des liens symboliques.
src/filters.c / include/filters.h	Application des filtres PID, utilisateur, commande, chemin et FD.
src/output.c / include/output.h	Mise en forme des résultats en tableau ou en mode PID seul.

src/utls.c / include/utls.h	Fonctions utilitaires : chemins, erreurs, conversions, nettoyage mémoire.
tests/	Scripts de tests et fichiers de comparaison.
Makefile	Compilation, nettoyage et exécution des tests.
README.md	Installation, usage, exemples, limites et captures de démonstration.

8.1 Structures de données envisagées

```
typedef struct {
    int show_header;
    int silent;
    int terse;
    pid_t pid_filter;
    char *user_filter;
    char *command_filter;
    char *path_filter;
} Options;
```

```
typedef struct {
    pid_t pid;
    char command[256];
    char user[64];
} ProcessInfo;
```

```
typedef struct {
    char fd[32];
    char type[32];
    char name[PATH_MAX];
    unsigned long inode;
    long long size;
} FdInfo;
```

8.2 Logique générale de traitement

1. Lire et valider les options de la commande.
2. Ouvrir le répertoire /proc.
3. Identifier les entrées numériques correspondant à des PID.
4. Pour chaque PID, récupérer les informations du processus.
5. Appliquer les filtres globaux : PID, utilisateur, commande.
6. Ouvrir /proc/[pid]/fd et parcourir les descripteurs.
7. Résoudre chaque lien symbolique avec readlink().
8. Récupérer les métadonnées utiles avec stat() ou lstat().
9. Appliquer les filtres liés au fichier ou au descripteur.
10. Afficher la ligne selon le format choisi.
11. Libérer les ressources et fermer les répertoires.

9. Stratégie de tests et validation

Les tests doivent prouver que mylsof fonctionne sur des situations simples et réalistes. Ils doivent aussi montrer que les erreurs courantes sont maîtrisées.

Type de test	Exemple	Validation attendue
Test de compilation	make puis make clean	Le projet compile sans erreur et se nettoie correctement.
Test d'aide	./mylsof --help	L'aide s'affiche avec les options et exemples.
Test de version	./mylsof --version	La version, les auteurs et le dépôt apparaissent.
Test sans option	./mylsof	Un tableau de résultats apparaît sans crash.

CAHIER DES CHARGES - PROJET MYLSOF

Test PID	<code>./mylsof -p \$\$</code>	Seuls les fichiers du processus demandé sont affichés.
Test utilisateur	<code>./mylsof -u \$USER</code>	Les résultats concernent l'utilisateur courant.
Test fichier	ouvrir un fichier puis lancer <code>./mylsof</code> chemin	Le processus ayant ouvert le fichier est retrouvé.
Test permissions	lancer sans sudo sur un système avec processus root	Les permissions refusées sont gérées proprement.
Comparaison indicative	<code>lsuf -p PID</code> puis <code>./mylsof -p PID</code>	Les résultats principaux sont cohérents, même si pas strictement identiques.
Test mémoire	<code>valgrind ./mylsof -p \$\$</code>	Aucune fuite majeure n'est détectée dans les chemins de base.

9.1 Critères de validation technique

- Le programme compile avec gcc sans erreur bloquante.
- Le programme s'exécute sans segmentation fault sur les cas de test prévus.
- Les erreurs de permission ou de fichiers disparus sont gérées.
- Les options prioritaires fonctionnent correctement.
- Le README permet à une autre personne de compiler et tester le projet.
- La démonstration montre au moins trois scénarios : sans option, filtrage par PID, recherche d'un fichier ouvert.

10. Livrables attendus

Les livrables servent à prouver que le projet est compris, développé, testé et présentable. Ils ne se limitent pas au code source : la documentation et la démonstration comptent autant pour valoriser le travail.

Livrable	Format attendu	Contenu minimum	Échéance recommandée
Cahier des charges	.docx et/ou .pdf	Contexte, objectifs, périmètre, contraintes, livrables, planning, références.	Début du projet
Dossier d'analyse et de conception	.docx ou .pdf	Architecture, modules, structures, algorithmes, choix techniques.	Après conception
Code source mylsof	Dépôt GitHub + archive .zip	Fichiers <code>.c/.h</code> , Makefile, organisation <code>src/include/tests</code> .	Fin développement
README utilisateur	README.md	Installation, compilation, options, exemples, limites connues.	Avant démonstration
Scripts de tests	.sh ou dossier tests	Cas de test simples, comparaison indicative avec <code>lsuf</code> , tests d'erreurs.	Phase tests
Rapport technique	.docx ou .pdf	Explication du fonctionnement, difficultés rencontrées, résultats, captures.	Fin projet
Présentation orale	.pptx ou PDF	Slides de présentation : contexte, architecture, démonstration, bilan.	Jour soutenance
Démonstration	Terminal + captures	Exécution de <code>mylsof</code> sur plusieurs commandes et cas réels.	Jour soutenance
Lien GitHub final	URL	Dépôt propre, commits lisibles, version finale taguée si possible.	Fin projet

11. Plan de travail

Le plan de travail ci-dessous est proposé sur six semaines. Il peut être raccourci ou étendu selon les dates réelles de remise. L'important est de garder une progression logique : comprendre, concevoir, coder, tester, documenter, présenter.

Phase	Durée estimée	Travaux à réaliser	Livrables intermédiaires
1. Cadrage et documentation	Semaine 1	Étudier <code>lsuf</code> , <code>/proc</code> et les fonctions C utiles. Définir les options prioritaires. Finaliser le cahier des charges.	Cahier des charges validé, liste des fonctionnalités.

CAHIER DES CHARGES - PROJET MYLSOF

2. Analyse et conception	Semaine 1 à 2	Définir l'architecture du programme, les structures de données et le format de sortie. Préparer le dépôt GitHub.	Dossier de conception, squelette du dépôt, Makefile initial.
3. Développement du noyau	Semaine 2 à 3	Coder le parcours de /proc, la détection des PID, la lecture de comm/status et la lecture de /proc/[pid]/fd.	Prototype capable d'afficher PID, commande, FD et nom de fichier.
4. Ajout des options prioritaires	Semaine 3 à 4	Implémenter --help, --version, -p, -u, -c, filtre par chemin, mode silencieux et mode PID seul.	Version fonctionnelle mylsof v0.8.
5. Tests et corrections	Semaine 4 à 5	Tester sur plusieurs processus, comparer avec lsof, corriger les erreurs de permissions, vérifier la mémoire.	Scripts de tests, corrections, version stable v1.0.
6. Documentation finale	Semaine 5 à 6	Rédiger le README, le rapport technique, préparer les captures, organiser la démonstration.	README, rapport, slides, dépôt GitHub propre.
7. Présentation et soutenance	Fin semaine 6	Présenter le contexte, l'architecture, les difficultés et faire une démonstration en direct.	Présentation orale et démonstration finale.

11.1 Mini-Gantt simplifié

Tâche	S1	S2	S3	S4	S5	S6
Recherche et cahier des charges	•	•				
Conception technique		•	•			
Développement noyau /proc			•	•		
Options et filtres				•	•	
Tests et corrections					•	•
Documentation et README					•	•
Présentation et démonstration						•

12. Critères d'acceptation

Le projet pourra être considéré comme réussi si les critères suivants sont respectés.

Critère	Condition d'acceptation
Compilation	make génère un exécutable mylsof sans erreur.
Aide	./mylsof --help affiche une aide claire et utilisable.
Affichage de base	./mylsof affiche plusieurs fichiers ouverts sur une machine Linux.
Filtre PID	./mylsof -p PID limite correctement les résultats au processus demandé.
Filtre utilisateur	./mylsof -u USER limite correctement les résultats à l'utilisateur demandé.
Gestion d'erreurs	Les permissions refusées, processus disparus et liens invalides ne provoquent pas de crash.
Documentation	Le README permet la compilation et l'exécution sans explication orale supplémentaire.
Démonstration	La soutenance montre un scénario réel : fichier ouvert, PID retrouvé, comparaison avec lsof.
Code	Les fichiers sont organisés, commentés quand nécessaire et présents sur GitHub.

13. Risques du projet et mesures de maîtrise

Risque	Impact	Mesure de maîtrise
Permissions refusées dans /proc	Résultats incomplets ou messages nombreux.	Gérer errno proprement, ajouter --silent, documenter la limite.
Processus qui disparaissent pendant le scan	Fichiers introuvables, erreurs intermittentes.	Tester chaque ouverture, ignorer les entrées devenues invalides.
Complexité de lsof original	Projet trop large.	Limiter le périmètre aux options prioritaires et assumer les exclusions.
Problèmes de buffers avec readlink()	Chemins tronqués ou bugs mémoire.	Utiliser PATH_MAX ou allocation contrôlée, vérifier la longueur retournée.
Affichage illisible	Démonstration moins convaincante.	Créer un module output.c et tester plusieurs tailles de chaînes.
Manque de temps pour les bonus	Fonctionnalités incomplètes.	Livrer d'abord le noyau stable, puis ajouter les bonus après validation.

Différences avec lsof officiel

Comparaison difficile.

Présenter mylsof comme une version pédagogique, pas comme un remplacement complet.

14. Références

Les références ci-dessous servent à justifier les choix techniques et à guider le développement. Elles doivent être consultées pendant la phase d'analyse et citées dans le rapport technique final.

[R1] lsof(8) - Linux manual page : <https://man7.org/linux/man-pages/man8/lsof.8.html>

[R2] proc(5) - Linux manual page : <https://man7.org/linux/man-pages/man5/proc.5.html>

[R3] proc_pid_fd(5) - Linux manual page : https://man7.org/linux/man-pages/man5/proc_pid_fd.5.html

[R4] proc_pid_cmdline(5) - Linux manual page : https://man7.org/linux/man-pages/man5/proc_pid_cmdline.5.html

[R5] Documentation officielle Linux Kernel - The /proc Filesystem : <https://docs.kernel.org/filesystems/proc.html>

[R6] readdir(3) - Linux manual page : <https://man7.org/linux/man-pages/man3/readdir.3.html>

[R7] readlink(2) - Linux manual page : <https://man7.org/linux/man-pages/man2/readlink.2.html>

[R8] stat(2) - Linux manual page : <https://man7.org/linux/man-pages/man2/stat.2.html>

[R9] getopt_long(3) - Linux manual page : https://man7.org/linux/man-pages/man3/getopt_long.3.html

[R10] getpwnam/getpwuid(3) - Linux manual page : <https://man7.org/linux/man-pages/man3/getpwnam.3.html>

[R11] Document fourni comme modèle : Cahier de charges - Projet d'implémentation de la commande grep en langage C, structure utilisée comme inspiration.

Annexes

Annexe A - Commandes utiles pendant le développement

Objectif	Commande
Compiler	make
Nettoyer	make clean
Exécuter sans option	./mylsof
Afficher l'aide	./mylsof --help
Tester le processus courant	./mylsof -p \$\$
Comparer avec lsof	lsof -p \$\$
Créer un fichier ouvert pour test	tail -f test.txt & puis ./mylsof test.txt
Tester la mémoire	valgrind ./mylsof -p \$\$

Annexe B - Exemple de README attendu

- Présentation rapide du projet mylsof.
- Prérequis : Linux, GCC, Make.
- Installation : git clone, cd mylsof, make.
- Utilisation : syntaxe générale et options disponibles.
- Exemples de commandes et captures de sortie.
- Limites connues : permissions, sockets réseau, compatibilité non Linux.
- Auteurs et lien vers le dépôt GitHub.

Annexe C - Définition de fini

Une fonctionnalité sera considérée comme finie lorsqu'elle sera codée, compilée, testée manuellement, documentée dans le README et montrable en démonstration sans manipulation spéciale.